

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке
Вариант №7

Студент гр. 4343

Васильев А. С.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Задача 1.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$). Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$.

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i p . Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Задача 2.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcah$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Задача варианта

Сделать вывод графического представления автомата Ахо-Корасик.

Теоретические сведения

Алгоритм Ахо — Корасика предназначен для поиска набора образцов в тексте за линейное время относительно длины текста, суммарной длины образцов и количества найденных вхождений.

Основная идея алгоритма заключается в том, что все образцы сначала помещаются в бор — префиксное дерево. Вершины бора соответствуют префиксам образцов, а рёбра помечены символами алфавита. Общие префиксы разных образцов хранятся один раз, что позволяет строить общий автомат для всего набора шаблонов.

После построения бора для его вершин вычисляются суффиксные ссылки. Суффиксная ссылка из вершины ведёт в вершину, соответствующую наибольшему собственному суффиксу текущей строки, который является префиксом какого-либо образца. Благодаря этим ссылкам при несовпадении символов алгоритм не возвращается назад по тексту, а переходит к подходящему состоянию автомата.

Также в автомате используются конечные ссылки. Конечная ссылка ведёт к ближайшей вершине по цепочке суффиксных ссылок, в которой заканчивается некоторый образец. Это позволяет эффективно находить все образцы, которые заканчиваются в текущей позиции текста, не проходя каждый раз по всей цепочке суффиксных ссылок.

После построения автомата текст просматривается один раз слева направо. При чтении очередного символа выполняется переход автомата в новое состояние. Если в текущей вершине или в вершинах, достижимых по конечным ссылкам, заканчиваются образцы, соответствующие вхождения добавляются в ответ.

Описание алгоритмов

Задача 1

В первой задаче программа сначала считывает текст, количество образцов и сами образцы. Для каждого образца сохраняется его длина, так как при поиске автомат находит позицию конца вхождения, а в ответе нужно вывести позицию начала.

После чтения входных данных создаётся автомат Ахо — Корасика. Каждый образец добавляется в бор посимвольно. Если нужного перехода из текущей вершины ещё нет, создаётся новая вершина. Когда весь образец добавлен, в последней вершине сохраняется номер этого образца. Таким образом, после добавления всех образцов в боре отмечены все терминальные вершины.

Затем выполняется построение автомата. Для этого используется обход в ширину. Сначала обрабатываются переходы из корня: отсутствующие переходы заменяются переходом обратно в корень, а существующие вершины добавляются в очередь. Далее для каждой вершины строится суффиксная ссылка, конечная ссылка и заполняются отсутствующие переходы. Если из вершины нет перехода по некоторому символу, он заменяется переходом из вершины, в которую ведёт суффиксная ссылка. После этого из любого состояния автомата можно выполнить переход по любому символу алфавита.

Поиск выполняется одним проходом по тексту. Для каждого символа программа переходит в новое состояние автомата. Если в текущей вершине заканчивается один или несколько образцов, программа фиксирует найденные вхождения. После этого дополнительно просматриваются вершины, достижимые по конечным ссылкам, так как в той же позиции текста могут заканчиваться образцы, являющиеся суффиксами текущего совпадения.

Для каждого найденного образца вычисляется позиция начала $start = end - |p_i| + 1$.

Здесь end — позиция конца найденного образца, а $|p_i|$ — его длина. Найденные номера образцов добавляются в массив ответов по позиции начала.

После завершения поиска программа проходит по этому массиву слева направо, сортирует номера образцов для каждой позиции и выводит пары в требуемом порядке.

Пусть $n = |T|$ — длина текста, S — суммарная длина всех образцов, а p — количество образцов. Построение бора занимает $O(S)$. Построение автомата занимает $O(S \cdot |\Sigma|)$, но в данной задаче алфавит фиксирован и состоит из пяти символов, поэтому эта часть имеет сложность $O(S)$. Сам проход по тексту занимает $O(n)$, но в одной позиции может быть найдено несколько образцов. В худшем случае в каждой позиции может заканчиваться до p образцов, поэтому обработка всех найденных вхождений оценивается как $O(n \cdot p)$. Итоговая временная сложность в худшем случае — $O(S + n \cdot p)$.

Память используется для хранения автомата и массива ответов. Автомат занимает $O(S)$ памяти при фиксированном алфавите. С учётом хранения всех найденных вхождений в худшем случае дополнительная память составляет $O(S + n \cdot p)$.

Задача 2

Во второй задаче программа сначала считывает текст, шаблон и символ джокера. Если длина шаблона больше длины текста, поиск сразу завершается, так как вхождение невозможно.

Далее шаблон разбивается на фрагменты, не содержащие джокер. Разбиение выполняется одним проходом по шаблону. Для каждого фрагмента сохраняется сама строка фрагмента и его смещение относительно начала шаблона. Например, если фрагмент начинается с позиции `offset`, то при нахождении этого фрагмента в тексте можно будет восстановить предполагаемое начало полного шаблона.

После разбиения все фрагменты добавляются в автомат Ахо — Корасика как отдельные образцы. Для каждого фрагмента отдельно сохраняются его длина и смещение. Затем автомат строится так же, как в первой задаче: создаются суффиксные ссылки, конечные ссылки и заполняются переходы автомата.

После построения автомата программа выполняет поиск фрагментов в тексте. Когда некоторый фрагмент найден, сначала вычисляется позиция начала этого фрагмента в тексте. Затем по сохранённому смещению вычисляется возможная позиция начала всего шаблона $full_start = fragment_start - offset$.

Если полученная позиция находится в допустимых границах, для неё увеличивается счётчик совпавших фрагментов. После обработки всего текста программа просматривает массив счётчиков. Если для некоторой позиции счётчик равен количеству фрагментов без джокера, значит, все обязательные части шаблона встретились на нужных местах, и эта позиция выводится как начало вхождения.

Пусть $n = |T|$ — длина текста, $m = |P|$ — длина шаблона, а r — количество фрагментов шаблона без джокера. Разбиение шаблона выполняется за $O(m)$. Суммарная длина всех фрагментов не превосходит m , поэтому построение бора и автомата занимает $O(m)$ при фиксированном алфавите. Поиск выполняется одним проходом по тексту, но в одной позиции может заканчиваться несколько фрагментов. В худшем случае их количество не превосходит r , поэтому обработка найденных фрагментов занимает $O(n \cdot r)$. Дополнительный проход по массиву счётчиков занимает $O(n)$.

Итоговая временная сложность составляет $O(m + n \cdot r)$.

Так как $r \leq m$, в худшем случае сложность не превосходит $O(n \cdot m)$.

Дополнительная память используется для автомата, массивов длин и смещений фрагментов, а также массива счётчиков возможных позиций начала шаблона. Поэтому дополнительная память составляет $O(n + m)$.

Задание варианта

Для индивидуального задания был реализован вывод графического представления автомата Ахо — Корасика для первой задачи. После построения бора программа сохраняет информацию о вершинах и рёбрах бора,

а после построения суффиксных и терминальных ссылок формирует файл в формате .dot, который можно визуализировать с помощью Graphviz.

В графе отображается сжатое представление построенного автомата: вершины, рёбра бора, суффиксные ссылки и терминальные ссылки. Каждая вершина подписывается своим номером и строкой, соответствующей текущему состоянию автомата. Вершины, в которых заканчиваются образцы, выделяются двойной окружностью, а внутри таких вершин указывается список out с номерами соответствующих образцов.

Рёбра бора обозначаются сплошными стрелками. Суффиксные ссылки обозначаются пунктирными рёбрами с подписью suf, а терминальные ссылки — точечными рёбрами с подписью term. Такое представление позволяет наглядно увидеть структуру бора, связи между состояниями автомата и вершины, соответствующие найденным образцам. Пример компактного графического представления автомата приведён на рисунке 1. Дополнительно формируется полное графическое представление автомата, в котором отображаются все переходы next по символам алфавита после построения автомата. Пример полного графического представления автомата на рисунке 2.

Сложность построения графического представления составляет $O(V)$ при фиксированном алфавите, где V – количество вершин автомата.

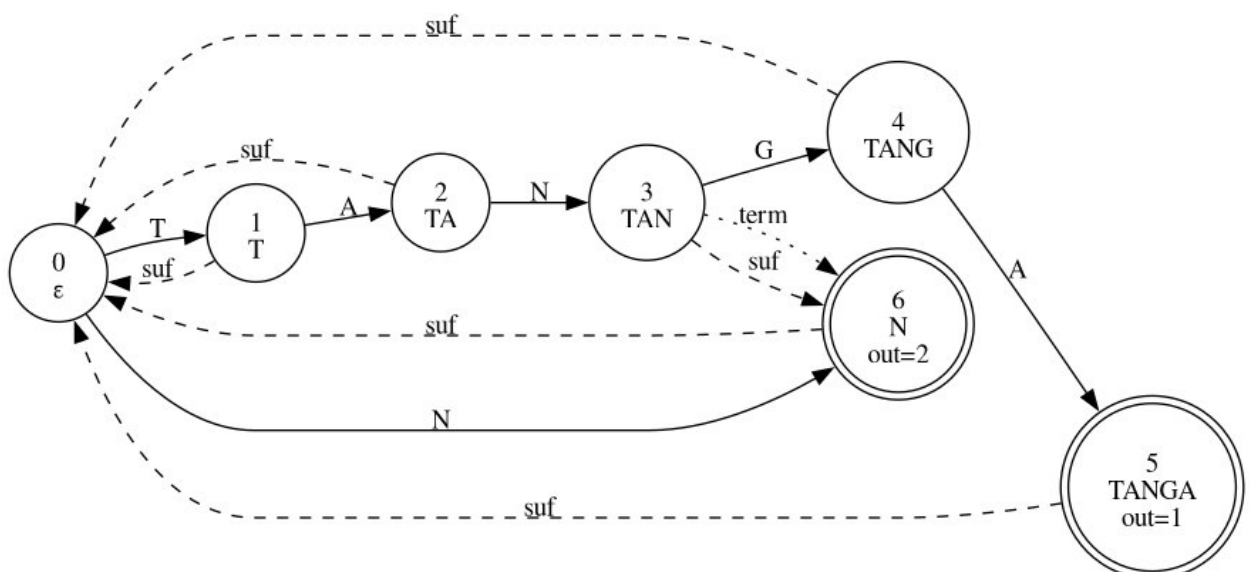


Рисунок 1 – Компактное графическое представление автомата Ахо-Корасик

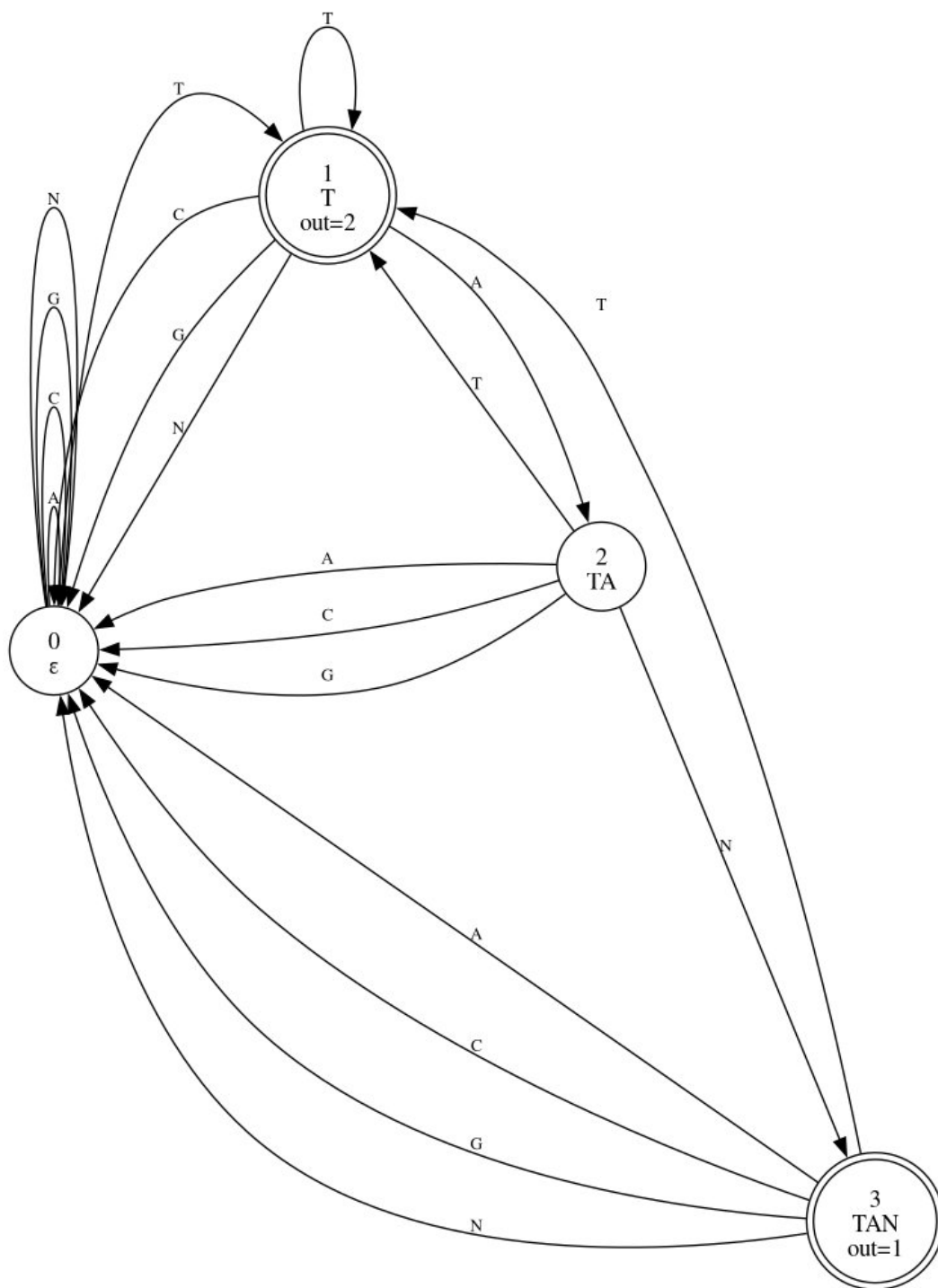


Рисунок 2 – Полное графическое представление автомата Ахо-Корасик

Тестирование

Программа	Входные данные	Выходные данные
Задача 1	NTAG 3 TAGT TAG T	2 2 2 3
	AAAA 2 A AA	1 1 1 2 2 1 2 2 3 1 3 2 4 1
	ACGTACGT 3 ACG CGT GT	1 1 2 2 3 3 5 1 6 2 7 3
Задача 2	ACGTACGA A?G ?	1 5
	ACGTTACNNT AC??T ?	1 6
	ACGTACGT N?N ?	вывод отсутствует